Constellation: A Bandwidth-Oriented Execution Model for Mixture-of-Experts Inference

Anonymous Authors

Abstract

We propose an attention–FFN disaggregation architecture that enables DeepSeek-V3 MoE inference with FP8-quantized shared expert layers on a $70,000 consumer-grade GPU cluster. Our system achieves 3000 tokens/s throughput with 20 ms p90 latency at 80% system utilization. The Mixture-of-Experts (MoE) architecture naturally aligns with a near-memory execution model. Under appropriate system orchestration, consumer GPUs serve as effective near-memory compute devices. Our system exhibits near-linear throughput scaling as GPUs are added and does not require specialized interconnects; low latency can be maintained using commodity Ethernet. Scheduling is extremely simple.

1 Introduction

Running large language models (LLMs) locally remains challenging due to their extreme memory bandwidth and compute requirements. At the same time, local inference is highly desirable because it provides stronger privacy guarantees, lower latency, and independence from cloud infrastructure. However, modern LLMs often contain tens to hundreds of billions of parameters, making efficient deployment on consumer hardware difficult.

Recent systems attempt to enable local inference of large models. For example, KTransformers runs very large models locally through CPU–GPU heterogeneous execution and memory offloading. However, such approaches remain limited by memory bandwidth, resulting in low throughput and high latency, particularly for Mixture-of-Experts (MoE) models where expert layers introduce substantial memory traffic.

In this work, we propose an attention–FFN disaggregation architecture for efficient MoE inference on consumer GPU clusters. To overcome memory

bandwidth limitations, we employ a large array of consumer GPUs to host MoE experts, representing the first use of a large consumer GPU cluster to serve sparse expert layers. Experts of each layer are evenly distributed across GPUs, making worst-case expert congestion unlikely. Even when imbalance occurs, its latency impact is amortized across layers along the token execution path, preventing any single layer from dominating end-to-end latency.

The key idea of this work is to treat consumer GPUs as near-memory compute devices for MoE experts. We adopt layer-parallel expert placement, distributing the experts of each layer across many GPUs. This spreads the memory bandwidth demand of expert layers across devices and avoids the bandwidth bottleneck that limits prior local inference systems.

A second observation is that expert imbalance primarily manifests as tail latency. Under layer-parallel expert placement, worst-case expert congestion becomes unlikely. Even when imbalance occurs, its latency impact is amortized across layers as tokens progress through the network.

Finally, we combine layer-parallel expert placement with attention–FFN disaggregation, executing attention layers on a small set of GPUs while distributing expert execution across many devices. A lightweight greedy scheduler routes tokens to available experts with minimal coordination, enabling low latency while scaling across many GPUs.

We demonstrate this architecture by running DeepSeek-V3 MoE inference on a $70,000 consumer GPU cluster, achieving 3000 tokenss throughput with 20 ms p90 latency at 80% system utilization using only commodity Ethernet interconnects.

Together, these ideas transform the bandwidth bottleneck of MoE inference into a scalable multi-GPU execution problem.

Contributions. This paper makes the following contributions:

- We identify the practical significance of attention–FFN disaggregation for large-scale MoE inference, showing that it enables expert execution to scale across a large array of consumer GPUs.
- We design a system that evenly distributes experts of each layer across many GPUs with replicated experts, spreading the memory bandwidth demand of MoE inference across devices.
- We develop a lightweight scheduling mechanism that greedily assigns tokens to expert replicas with the earliest available execution slot, enabling low-latency inference without complex global coordination.

2 Background

2.1 Transformer Inference

Transformer models perform inference sequentially across layers. Each layer typically consists of a self-attention module followed by a feed-forward network (FFN). During autoregressive decoding, tokens are processed layer by layer, and the computation of each layer must complete before the next layer begins.

In dense Transformer models, FFN layers dominate the computational cost of inference due to their large matrix multiplications and parameter sizes. As model sizes grow, these FFN layers become increasingly expensive to execute and store, motivating architectures that improve efficiency while preserving model capacity.

2.2 Mixture-of-Experts Models

Mixture-of-Experts (MoE) models replace dense FFN layers with a set of experts [**?**, **?**, **?**]. For each token, a routing function selects a small subset of experts to process the token. The number of selected experts varies across models; some systems use top-1 or top-2 routing, while larger MoE models such as DeepSeek-V3 route tokens to multiple experts (e.g., top-8) [**?**]. This sparse activation allows MoE models to significantly increase model capacity while keeping the per-token computation relatively small.

However, MoE layers introduce additional system challenges. Each routed token must access the parameters of the selected experts, which can involve substantial parameter movement. As the number and size of experts grow, expert execution increasingly becomes *memory-bandwidth bound* rather than compute bound.

2.3 Challenges in MoE Inference

Efficient inference of large MoE models presents two key system challenges.

First, expert execution introduces substantial **memory bandwidth demand**. In large MoE models, expert parameters often exceed the effective memory bandwidth of a single GPU, limiting throughput even when sufficient compute resources are available. This issue is particularly pronounced in *local inference* settings, where batching opportunities are limited and bandwidth bottlenecks are harder to hide.

Second, MoE routing is inherently **imbalanced**. Prior work has shown that token-to-expert assignments often exhibit highly skewed distributions,

where a small subset of experts receives a disproportionate number of tokens while others remain underutilized [**?**, **?**]. Such imbalance can create overloaded experts that dominate the completion time of a layer, increasing tail latency during inference.

These challenges motivate system designs that distribute expert execution across multiple devices while maintaining efficient token routing and low end-to-end latency.

3 System Overview

Our system separates attention computation from expert execution and distributes expert execution across a large pool of GPUs.

Figure 1 illustrates the architecture. Attention layers are executed on a small set of attention GPUs that maintain the KV cache. After each attention computation, tokens are processed by the MoE router, which selects the target experts according to the model routing function.

Experts are hosted on a large pool of GPUs. Instead of placing all experts of a layer on a single device, experts of the same layer are evenly placed across multiple GPUs. Each expert may also be replicated across multiple GPUs.

After routing, tokens are passed to a lightweight scheduler that assigns each token to a replica of the selected expert. The scheduler greedily selects the replica with the earliest slot.

The scheduler does not attempt to avoid assigning multiple experts of the same token to the same GPU, as doing so would require additional coordination and increase scheduling latency.

After expert execution, outputs are returned to the attention GPUs to continue processing in the next layer.

4 Architecture

The design of our system is primarily driven by latency considerations. In autoregressive inference, tokens must pass through all layers sequentially, and delays at any layer directly increase end-to-end latency.

For MoE models such as DeepSeek-V3, each layer may activate multiple experts for a token. When expert execution becomes congested, tokens may experience significant queueing delays, which dominate overall latency.

To control latency, our system adopts a simple scheduling policy that assigns tokens to the earliest available expert replica. Interestingly, this

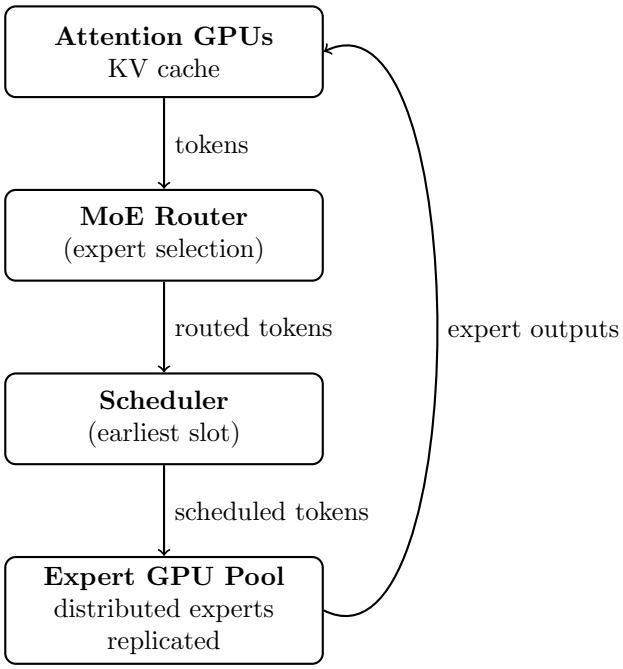

Figure 1: System overview. Tokens are first processed by attention GPUs that maintain the KV cache. They are then routed by the MoE router and scheduled to expert replicas across a large pool of expert GPUs. Expert outputs are returned to the attention GPUs to continue execution in the next layer.

latency-driven policy naturally spreads the workload across many GPUs, allowing the system to utilize the combined memory bandwidth of multiple devices without explicitly optimizing for bandwidth balancing.

We now develop an analytical model for MoE inference latency. The model is intentionally simple: it isolates the effects of routing skew, replica counts, and dispatch policy on queue imbalance and latency tail.

4.1 Latency Imbalance under Routing Skew

We first analyze a single MoE layer under random dispatch. This baseline isolates how routing skew and replica placement induce load imbalance before introducing load-aware scheduling.

Consider one MoE layer executed on G expert GPUs. Expert e is repli-

cated on a subset of GPUs

$$R_e \subseteq \{1, \ldots, G\}, \qquad |R_e| = r_e.$$

Each token activates k experts. Let p_e denote the expected number of selections of expert e per token, so that

$$\sum_{e=1}^{E} p_e = k.$$

Let Λ be the token arrival rate. Then the total request arrival rate of expert e is Λp_e.

Under random dispatch, a request for expert e is sent uniformly to one of its r_e replicas, i.e.,

$$q_{g,e} = \begin{cases} \frac{1}{r_e}, & g \in R_e, \\ 0, & g \notin R_e. \end{cases}$$

Hence the arrival rate at GPU g is

$$\lambda_g = \Lambda \sum_{e:\, g \in R_e} \frac{p_e}{r_e}.$$

We assume each expert request has deterministic service time τ and service rate

$$\mu = \frac{1}{\tau}.$$

The utilization of GPU g is

$$\rho_g = \lambda_g \tau.$$

A token completes the layer only after all of its k expert requests finish. Let W_j denote the queueing delay of the j-th expert request. Then the layer latency is

$$T = C + \max_{j=1,\ldots,k} W_j,$$

where C absorbs all constant per-token overheads. Approximating each GPU as an $M/D/1$ queue,

$$\mathbb{E}[W_g] = \frac{\lambda_g \tau^2}{2(1-\rho_g)} = \frac{\rho_g}{2\mu(1-\rho_g)}.$$

Thus latency is dominated by the busiest GPUs.

To quantify how such hot GPUs emerge, assume each replica set R_e is placed uniformly at random among all subsets of size r_e. Let

$$X_{g,e} = \mathbf{1}[g \in R_e].$$

Then

$$\lambda_g = \Lambda \sum_{e=1}^{E} \frac{p_e}{r_e} X_{g,e}.$$

Proposition 1 (Random-dispatch imbalance)**.** *Under random dispatch and independent random replica placement across experts,*

$$\mathbb{E}[\lambda_g] = \frac{k\Lambda}{G},$$

and

$$\mathrm{Var}(\lambda_g) = \frac{\Lambda^2}{G} \sum_{e=1}^{E} \frac{p_e^2}{r_e} \left(1 - \frac{r_e}{G}\right).$$

In particular, when $G \gg r_e$,

$$\mathrm{Var}(\lambda_g) \approx \frac{\Lambda^2}{G} \sum_{e=1}^{E} \frac{p_e^2}{r_e}.$$

Proof. Since $\Pr(X_{g,e} = 1) = r_e/G$,

$$\mathbb{E}[\lambda_g] = \Lambda \sum_{e=1}^{E} \frac{p_e}{r_e} \frac{r_e}{G} = \frac{\Lambda}{G} \sum_{e=1}^{E} p_e = \frac{k\Lambda}{G}.$$

Assuming independent replica placements across experts,

$$\mathrm{Var}(\lambda_g) = \Lambda^2 \sum_{e=1}^{E} \left(\frac{p_e}{r_e}\right)^2 \mathrm{Var}(X_{g,e}),$$

and $\mathrm{Var}(X_{g,e}) = \frac{r_e}{G}(1 - \frac{r_e}{G})$. Substituting yields the result. □

Proposition 1 shows that random-dispatch imbalance is governed by

$$\sum_{e=1}^{E} \frac{p_e^2}{r_e}.$$

Routing skew increases this quantity through large p_e, while replicating hot experts reduces it through r_e.

Using Proposition 1 to characterize fluctuation scale, we write

$$\lambda_{\max} \approx \frac{k\Lambda}{G} + O\left(\Lambda\sqrt{\frac{\log G}{G}\sum_{e=1}^{E}\frac{p_e^2}{r_e}}\right).$$

Therefore, under random dispatch, latency degradation is controlled not by average utilization alone, but by the imbalance term $\sum_e p_e^2/r_e$, which determines how much larger $\lambda_{\max}$ is than the cluster-average load.

4.2 Greedy Dispatch over Restricted Replica Sets

We next show how load-aware dispatch mitigates the imbalance above. The constraint remains unchanged: an expert-e request can only be dispatched to GPUs in R_e.

Let $S(t)$ denote the system state observed at request arrival time t. For an expert-e request, define

$$q_{g,e}(s) = \Pr(\text{dispatch to GPU } g \mid S(t) = s,\ \text{expert } e \text{ is selected}).$$

The long-run arrival rate at GPU g is

$$\lambda_g = \Lambda \sum_{e=1}^{E} p_e \bar{q}_{g,e}, \qquad \bar{q}_{g,e} := \mathbb{E}[q_{g,e}(S(t))].$$

Thus dispatch affects latency through the induced routing fractions $\bar{q}_{g,e}$.

Under random dispatch,

$$\bar{q}_{g,e}^{\text{rand}} = \frac{1}{r_e}, \qquad g \in R_e.$$

We instead consider a greedy policy that always dispatches an expert request to the shortest eligible queue. Since each request has deterministic service time, queue length and workload are equivalent up to tie-breaking and residual-service effects, so queue length is a sufficient dispatch metric.

Let $Q_g(s)$ denote the queue length of GPU g in state s, and define

$$M_e(s) = \arg\min_{h \in R_e} Q_h(s).$$

With uniform tie-breaking,

$$q_{g,e}^{\text{greedy}}(s) = \begin{cases} \frac{1}{|M_e(s)|}, & g \in M_e(s), \\ 0, & g \notin M_e(s). \end{cases}$$

Proposition 2 (One-step optimality of greedy dispatch)**.** *Fix a system state s and an arriving request for expert e. If the request is dispatched to GPU $j \in R_e$, the post-dispatch queue lengths are*

$$Q'_h(s; j) = Q_h(s) + \mathbf{1}[h = j], \qquad h \in R_e.$$

Let

$$j^* \in \arg\min_{h \in R_e} Q_h(s).$$

Then for any admissible choice $j \in R_e$,

$$\max_{h \in R_e} Q'_h(s; j^*) \leq \max_{h \in R_e} Q'_h(s; j).$$

That is, greedy dispatch minimizes the maximum post-dispatch queue length over the eligible replicas of expert e.

Proof. Dispatching the request to GPU j increases only $Q_j(s)$ by one. To minimize the resulting maximum over R_e, the increment should be applied to a GPU with minimum current queue length. Adding the request to any larger queue can only weakly increase the post-dispatch maximum. □

Relative to random dispatch, greedy dispatch does not change routing skew $\{p_e\}$ or replica placement $\{R_e\}$. Instead, it adaptively reshapes the routing fractions over each replica set R_e to avoid already congested replicas. Consequently, greedy dispatch suppresses the formation of heavily loaded GPUs, tends to reduce $\lambda_{\max}$ in steady state, and improves token latency tail behavior.

4.3 From Per-Layer Latency to End-to-End Token Latency

We now extend the single-layer analysis to the full model. Our goal is to estimate end-to-end token latency by summing latency contributions across layers.

Consider a model with L MoE layers indexed by $\ell = 1, \ldots, L$. For layer ℓ, let T_ℓ denote its latency contribution. Then the total token latency is

$$T_{\text{tot}} = C_{\text{dense}} + \sum_{\ell=1}^{L} T_\ell,$$

where C_{dense} aggregates all non-MoE latency contributions, including dense layers, attention, and fixed software overheads outside expert execution.

Per-layer decomposition. For each MoE layer ℓ, as in Section 4.1, the latency can be written as

$$T_\ell = C_\ell + \max_{j=1,\ldots,k} W_{\ell,j},$$

where C_ℓ is the constant part of layer-ℓ latency and $W_{\ell,j}$ is the queueing delay seen by the j-th expert request in that layer.

Therefore,

$$T_{\text{tot}} = C_{\text{dense}} + \sum_{\ell=1}^{L} C_\ell + \sum_{\ell=1}^{L} \max_{j=1,\ldots,k} W_{\ell,j}.$$

Define

$$C_{\text{tot}} = C_{\text{dense}} + \sum_{\ell=1}^{L} C_\ell.$$

Then

$$T_{\text{tot}} = C_{\text{tot}} + \sum_{\ell=1}^{L} \max_{j=1,\ldots,k} W_{\ell,j}.$$

Taking expectation gives the exact decomposition

$$\mathbb{E}[T_{\text{tot}}] = C_{\text{tot}} + \sum_{\ell=1}^{L} \mathbb{E}\left[\max_{j=1,\ldots,k} W_{\ell,j}\right].$$

Thus end-to-end latency is additive in expectation across layers.

Layer-wise latency estimate. For layer ℓ, let

$$\lambda_{\max,\ell} = \max_g \lambda_{\ell,g}$$

denote the maximum GPU arrival rate under the dispatch policy of interest, and let

$$\rho_{\max,\ell} = \frac{\lambda_{\max,\ell}}{\mu_\ell} = \lambda_{\max,\ell}\tau_\ell,$$

where τ_ℓ and $\mu_\ell = 1/\tau_\ell$ are the service time and service rate for one expert request at layer ℓ.

Using the $M/D/1$ approximation from Section 4.1, the mean queueing delay associated with the busiest GPU at layer ℓ is

$$\bar{W}_{\max,\ell} := \frac{\lambda_{\max,\ell}\tau_\ell^2}{2(1-\rho_{\max,\ell})} = \frac{\rho_{\max,\ell}}{2\mu_\ell(1-\rho_{\max,\ell})}.$$

Since the per-layer latency is controlled by the slowest among the k expert requests, a natural approximation is

$$\mathbb{E}\left[\max_{j=1,\dots,k} W_{\ell,j}\right] \approx \gamma_k \bar{W}_{\max,\ell},$$

where γ_k is an effective fan-out factor satisfying

$$1 \le \gamma_k \le k.$$

The lower end corresponds to strongly correlated queueing delays among the selected replicas, while the upper end corresponds to the worst case in which one sums k comparable delays. Substituting yields

$$\mathbb{E}[T_{\text{tot}}] \approx C_{\text{tot}} + \gamma_k \sum_{\ell=1}^{L} \frac{\rho_{\max,\ell}}{2\mu_\ell(1-\rho_{\max,\ell})}.$$

This expression shows that total latency grows roughly linearly with the number of MoE layers, but superlinearly as any layer approaches saturation.

A simple bound. The same decomposition also yields a coarse but useful bound. Since for any nonnegative random variables,

$$\max_{j=1,\dots,k} W_{\ell,j} \le \sum_{j=1}^{k} W_{\ell,j},$$

we have

$$\mathbb{E}\left[\max_{j=1,\dots,k} W_{\ell,j}\right] \le k\bar{W}_{\max,\ell}.$$

On the other hand,

$$\mathbb{E}\left[\max_{j=1,\dots,k} W_{\ell,j}\right] \ge \bar{W}_{\max,\ell}.$$

Therefore,

$$C_{\text{tot}} + \sum_{\ell=1}^{L} \bar{W}_{\max,\ell} \;\le\; \mathbb{E}[T_{\text{tot}}] \;\le\; C_{\text{tot}} + k\sum_{\ell=1}^{L} \bar{W}_{\max,\ell}.$$

Hence the end-to-end latency is sandwiched between one and k times the sum of the per-layer busiest-GPU queueing delays.

Homogeneous-layer simplification. If MoE layers are statistically similar, so that

$$\mu_\ell \approx \mu, \qquad \rho_{\max,\ell} \approx \rho_{\max}, \qquad \forall \ell,$$

then the total latency estimate simplifies to

$$\mathbb{E}[T_{\text{tot}}] \approx C_{\text{tot}} + L\gamma_k \frac{\rho_{\max}}{2\mu(1-\rho_{\max})}.$$

Thus the model predicts two basic scaling laws: latency grows approximately linearly with the number of MoE layers L, and grows sharply as the busiest-layer utilization $\rho_{\max}$ approaches 1.

Connection to routing skew and dispatch policy. Under random dispatch, Section 4.1 gives the heuristic estimate

$$\lambda^{\text{rand}}_{\max,\ell} \approx \frac{k\Lambda}{G} + O\left(\Lambda\sqrt{\frac{\log G}{G}\sum_{e=1}^{E}\frac{p_{\ell,e}^2}{r_{\ell,e}}}\right),$$

so the end-to-end latency is controlled by the layer-wise imbalance terms

$$\sum_{e=1}^{E}\frac{p_{\ell,e}^2}{r_{\ell,e}}.$$

Greedy dispatch does not change $p_{\ell,e}$ or $r_{\ell,e}$, but reduces effective load concentration and therefore tends to reduce $\lambda_{\max,\ell}$ at each layer. Consequently, its benefit accumulates across layers through the sum above.

5 System Simulation

We use a discrete-event simulator to validate the latency analysis under multi-layer execution. The simulator preserves restricted replica sets, skewed expert demand, FIFO expert service, and layer-by-layer token progression.

5.1 Setup and Results

We simulate G expert GPUs and L MoE layers. Each layer has E experts; each token activates k distinct experts; each expert is replicated on r GPUs with balanced placement. Token arrivals follow a Poisson process with rate Λ. At each layer, activated experts are sampled without replacement from a Zipf distribution with parameter α. Each GPU is modeled as a FIFO server with deterministic service time τ, and a token advances only after all k expert requests of the current layer complete. We add a fixed per-layer overhead C.

We compare **random dispatch**, which selects a replica uniformly at random, with **greedy dispatch**, which selects the eligible replica with the shortest current queue and adds a penalty when multiple requests from the same token and layer are assigned to the same GPU.

Unless otherwise stated, we use $G = 168$, $L = 58$, $E = 256$, $k = 8$, and $r = 4$. Each run simulates 5000 tokens. The default service time is $240\,\mu s$, the fixed per-layer overhead is $10\,\mu s$, the duplicate-placement penalty is $500\,\mu s$, and the default routing distribution is Zipf with $\alpha = 1.1$.

We sweep scheduler, replica count, arrival rate, and routing skew: scheduler $\in \{\text{random}, \text{greedy}\}$, $r \in \{2, 3, 4, 6\}$, $\Lambda \in \{500, 1000, 1500, 2000, 2500, 3000\}$ tokens/s, and $\alpha \in \{0.0, 0.8, 1.1\}$. We measure end-to-end token latency and throughput.

The simulation shows three consistent trends. First, random dispatch develops a much heavier latency tail as load increases, while greedy dispatch keeps p95 and p99 flatter by suppressing queue buildup within restricted replica sets. Second, greedy dispatch sustains higher throughput before saturation. Third, increasing the replica count r improves both schedulers, but more strongly for greedy dispatch, since replication both averages load and enlarges the scheduler's choice set.

figures/p95_latency_vs_arrival.pdf figures/throughput_vs_arrival.pdf figures/p99_latency_vs_arrival.pdf

Figure 2: Simulation results. Left: p95 token latency versus arrival rate. Middle: throughput versus arrival rate. Right: p99 token latency versus arrival rate. Greedy dispatch reduces tail latency and sustains higher throughput, with larger gains under stronger routing skew and larger replica sets.

The gap between the two schedulers also widens with routing skew. As α increases, hot experts become more pronounced and random dispatch degrades more quickly. Overall, latency is driven primarily by *queue imbalance* rather than mean load alone, and a simple restricted-set greedy scheduler captures most of the benefit of replication.

6 Prototype and Planned Evaluation

The next step of this work is to build a small-scale prototype of the proposed expert-disaggregated MoE inference system. The prototype will include a frontend router, a pool of expert workers with replicated experts, and a lightweight scheduler that dispatches each request within its restricted replica set.

Our goal is to test whether the main conclusions of the analytical model and simulator hold in a real deployment: whether greedy dispatch reduces tail latency, whether replication improves both throughput and robustness to routing skew, and whether these gains can be achieved on low-cost commodity hardware without specialized interconnects.

A full prototype implementation and measurement study are left to future work.

7 From Cloud Prototype to SSD-Based Product Design

Having evaluated the system on cloud GPU servers, we now turn to a concrete SSD-based product design. Our goal is not to maximize throughput on a single device, but to identify a compact SSD configuration with usable decoding latency for personal deployment, and then show how the same package design scales to server settings.

7.1 Baseline Device

Our baseline product is a single 4 TB SSD composed of 16 packages. Each package acts as a near-storage expert server: it stores many experts across many layers and serves one expert request at a time. We target an effective service time of approximately 1 ms per expert request.

Storage capacity is not the limiting factor. Even with expert replication, the full model footprint remains well within a 4 TB device. The package count is therefore chosen by service concurrency rather than by capacity. We use asymmetric replication: most experts use 4 replicas, while the top-32 hottest experts in each layer use 8 replicas.

7.2 Package Requirement

The 1 ms target imposes a concrete package-level requirement. For an expert of approximately 44 MB in FP8 format, a 1 ms service time requires an effective package-internal read bandwidth of about 44 GB/s.

Our baseline assumes 14-way die-level parallelism with an effective stream granularity of 32 KiB per die, yielding 448 KiB per effective read interval. Under an effective read latency of 9.7 μs, this corresponds to approximately 47 GB/s of package-internal streaming bandwidth, sufficient for the 1 ms target.

The arithmetic requirement is modest: roughly 50 GMAC/s is sufficient to sustain one expert request. The dominant local resource is buffering rather than compute. We therefore provision approximately 1–1.5 MiB of local SRAM per package for double- or triple-buffered weight streaming.

7.3 Performance Target

With 16 packages, the expert side can complete the 8 expert requests of a layer in a single round under light load. This yields an expert-side latency

target of approximately 58 ms/token for 58 sparse layers. Adding GPU-side attention and routing, together with modest control and transfer overhead, yields a total decoding latency target of roughly 70–75 ms/token.

For a single autoregressive decoding stream, the steady-state throughput is determined by the per-token latency:

$$\text{throughput}_{\text{single-stream}} \approx \frac{1}{T_{\text{token}}}.$$

At 70–75 ms/token, this corresponds to approximately 13–14 tokens/s for a single stream.

7.4 High-End Scaling

A higher-end deployment targets server-scale rather than client-scale use. Instead of using one or two SSDs, a server can host many such 4 TB, 16-package SSDs in parallel. This increases the number of package-level expert servers almost proportionally to the number of installed SSDs.

The main benefit of this configuration is not a lower single-stream latency floor, since the package-level service time remains unchanged, but much higher aggregate throughput. By installing many such SSDs, the system can support many concurrent decoding streams, and total throughput scales approximately linearly with the number of SSDs until another component, such as the GPU-side attention path or host-side control, becomes the bottleneck.

In this sense, the baseline device defines the package-level latency point, while the server configuration scales system throughput by replicating that point across many SSDs.

7.5 Product Position

This design point highlights the operating point of our architecture. The proposed SSD-based expert engine is not intended to compete with large GPU servers on raw per-device throughput. Instead, it targets low-cost, high-capacity, physically compact expert execution for personal or edge deployment, while still allowing server-scale aggregate throughput by installing many such devices in parallel.

As a concrete target, we set the baseline MSRP of this SSD to $1,199. This price point is above that of a conventional high-end 4 TB SSD, but remains plausible for a premium personal inference device while reflecting the added near-storage compute and package-level integration.

8 Broader Implications

Beyond system design, the proposed architecture may have broader implications for LLM research, deployment, business models, and hardware markets. If sparse expert execution becomes cheap enough for personal deployment, model ownership and adaptation may move away from centralized cloud serving.

8.1 Implications for LLM Research and Evaluation

If large MoE models become deployable on personal hardware, users may no longer interact with a single fixed hosted model, but with locally adapted descendants of a common base model. In this regime, model memory may move from external retrieval or session state into the parameters themselves, allowing persistent user-specific behavior to be encoded directly in expert weights. This may improve personalization, but it also makes model lineage, auditing, and reproducibility harder to track.

A similar shift applies to evaluation. If many deployed models are continuously modified through local expert updates, then standard evaluation on a single fixed checkpoint becomes less informative. Evaluation may increasingly need to account for families of related descendants, deployment-specific configurations, and long-term behavior after continued local adaptation. In this sense, the relevant object of evaluation may shift from a single static model to a changing population of personalized variants.

8.2 Implications for Deployment and Business Models

If sparse expert inference can be deployed efficiently on local hardware, then sensitive data no longer needs to be sent to a centralized inference provider by default. This may make local deployment particularly attractive in domains such as healthcare, law, finance, and enterprise internal systems. At the same time, it shifts part of the governance problem from service-side control to model distribution, update mechanisms, and device-side policy enforcement.

The same shift may also reshape open-source ecosystems and AI business models. If high-capacity MoE models become practical on personal hardware, open models would no longer be valuable only as research artifacts or limited local demos, but as systems that users can run, modify, and maintain over long periods. This could encourage a larger ecosystem of forks, expert variants, and locally adapted descendants, and shift competi-

tion from hosted API quality toward the long-term value of the base model and its surrounding community.

Under this deployment model, selling remote tokens may also become a less defensible business model for some classes of inference. Centralized providers remain important for training, global updates, and premium hosted services, but value may shift from metered inference toward base models, update pipelines, personalization tools, safety layers, and application ecosystems.

8.3 Implications for Hardware Markets

In the regime studied here, sparse expert inference is driven less by peak dense compute than by timely delivery of large expert weights. If this deployment model becomes important, storage-centric platforms and package-level memory-system design may capture a larger share of inference value, while GPUs remain important primarily for training and for the dense components of LLM inference.

9 Conclusion

MoE expert inference is entirely achievable with current technology; it simply requires assembling existing building blocks in a way that at first appears impossible.